Rafeeq – LLM Track Research

Milestone 1: Deaf & Mute · LLM Team

Prepared for the Friday session · Rafeeq browser-based accessibility extension

1. Scope of the LLM Track

In Milestone 1, the LLM side of the pipeline handles two jobs:

1. **Sign → Text (Grammar Correction).** The CV gesture classifier outputs raw glosses, i.e. isolated sign tokens such as `["يريد", "كشف", "مستشفى", "أنا"]`. Sign language does not follow spoken Arabic grammar (no prepositions, no verb conjugation, no connectors). The LLM turns these tokens into one fluent Arabic sentence, e.g. "أنا أريد الذهاب إلى المستشفى لإجراء كشف".
2. **Speech → Captions (Reverse Path).** The hearing user's speech is captured from the browser microphone, transcribed to Arabic text in real time, and shown as live captions for the deaf user.

2. Pre-trained Models Compared

Model	Role	Pros	Cons	Verdict
Google Gemini Flash	Gloss → Text	Very low latency, strong Arabic, free tier in Google AI Studio	Cloud API dependency	**Chosen**
OpenAI GPT-4o-mini	Gloss → Text	Fast, consistent output, good Arabic, cheap	No free tier, needs billing setup	Alternative
Llama 3.1 8B (Groq)	Gloss → Text	Extremely fast inference, free tier	Weaker Arabic grammar on complex cases	Fallback
OpenAI Whisper (Large-v3 / API)	Speech → Text	Top Arabic accuracy (MSA + dialects), robust to noise	STT only, needs an audio pipeline	**Chosen**

3. Selected Models and Why

A. Gloss → Text: Google Gemini Flash

- **Speed:** built for low-latency, high-throughput calls, which suits near real-time use in a browser extension.
- **Arabic quality:** handles Modern Standard Arabic and Egyptian dialect well, and can infer missing prepositions and verb conjugations from broken keywords.
- **Cost:** the free tier in Google AI Studio is enough to build, test, and demo the prototype at no cost.

B. Speech → Captions: OpenAI Whisper (or faster-whisper)

- **Accuracy:** noticeably fewer errors on conversational Arabic and regional accents than the browser's built-in Web Speech API.
- **Flexible hosting:** use the OpenAI Audio API, or run **faster-whisper** on our own backend for free.

4. API Details and Documentation

Component	SDK	Documentation
Gemini API (Gloss → Text)	`google-genai` (Python)	Gemini API Docs
Whisper API (Speech → Text)	`openai` (Python)	OpenAI Speech-to-Text Guide
faster-whisper (self-hosted option)	`faster-whisper` (Python)	GitHub Repository

Sample: Gloss → Text with Gemini

```
from google import genai
from google.genai import types

client = genai.Client(api_key="YOUR_GEMINI_API_KEY")

system_instruction = (
    "You are an Arabic Sign Language gloss-to-text translator. "
    "Take raw, fragmented sign tokens and output a single, natural, "
    "grammatically correct Arabic sentence. Do not add conversational
filler."
```

```
)

# Example input from the CV gesture classifier
raw_glosses = ["يحتاج" ,"دواء", "تعبان" ,"أنا"]

response = client.models.generate_content(
    model="gemini-2.5-flash",  # any current Flash model works
    contents=f"Glosses: {raw_glosses}",

config=types.GenerateContentConfig(system_instruction=system_instruction),
)
print(response.text.strip())
# Expected output: "أنا متعب وأحتاج إلى دواء."
```

5. Key Technical Points

- **Few-shot prompting:** include 3–5 fixed input/output examples in the system prompt so the model only rewrites the glosses and never replies conversationally or hallucinates extra content.
- **Buffer-based batching:** the CV side should not send every detected frame. The backend collects glosses over a 2–3 second window (or until a pause gesture) and sends one request to the LLM, which cuts latency and token usage.